

CS 340



Part 1 Overview

Agenda

1. Learning Goals
2. CS 340 Part 1 Overview
 - a. Layers of abstraction
 - b. “Hello world” at multiple levels
 - c. Our path forward

CS 340 - Computer Systems

LG: Understanding, at a high level, how we get from electricity to a website.

LG: Developing mental models of how different computer systems work so that you can understand more complex ideas if you would like.

LG: Getting experience working on unstructured, substantial coding projects.

340 Part 1 - Overview

Today's LG: To give context to the course material to start seeding concepts and how they fit into the big picture.

1. Layers of abstraction
2. “Hello world” at multiple levels
3. Our path forward

What I mean by abstraction



[Cheryl Harrison](#)

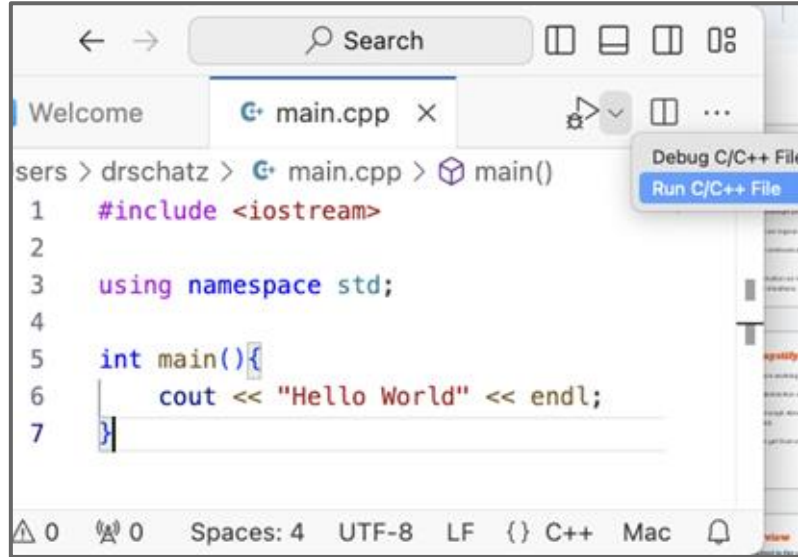
I drove to work

I opened the door, sat down, put radio....

I moved my left leg 2 cm forwards

*** The cell in my leg reacted

Computer Layers of Abstraction

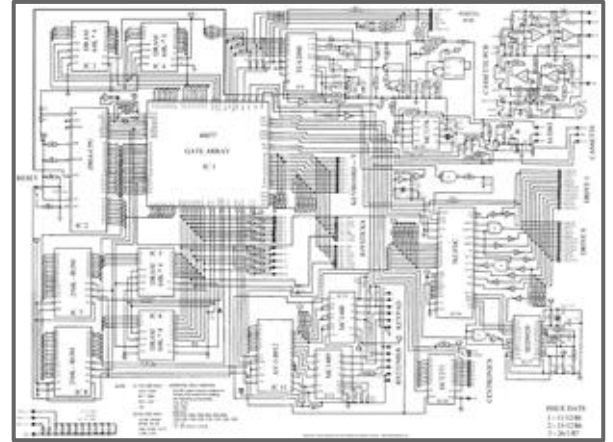


A screenshot of a C++ IDE window. The title bar shows 'Welcome' and 'main.cpp'. The code editor displays the following C++ code:

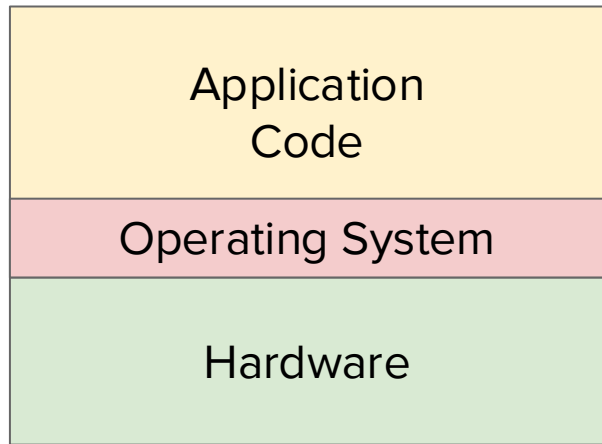
```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6     cout << "Hello World" << endl;
7 }
```

The IDE interface includes a search bar at the top, a breadcrumb trail 'users > drschatz > main.cpp > main()', and a status bar at the bottom showing 'Spaces: 4 UTF-8 LF {} C++ Mac'.

→ **CS**
340 ←



Big Ideas



↙ vscode, notes app, helloworld

↙ code that is run on hardware

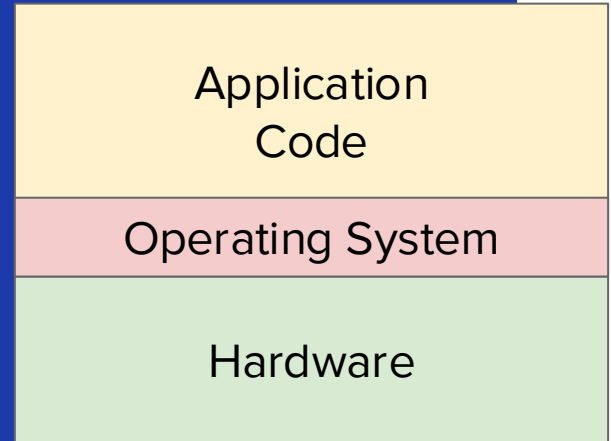
↙ transistors
gates

What should you remember?

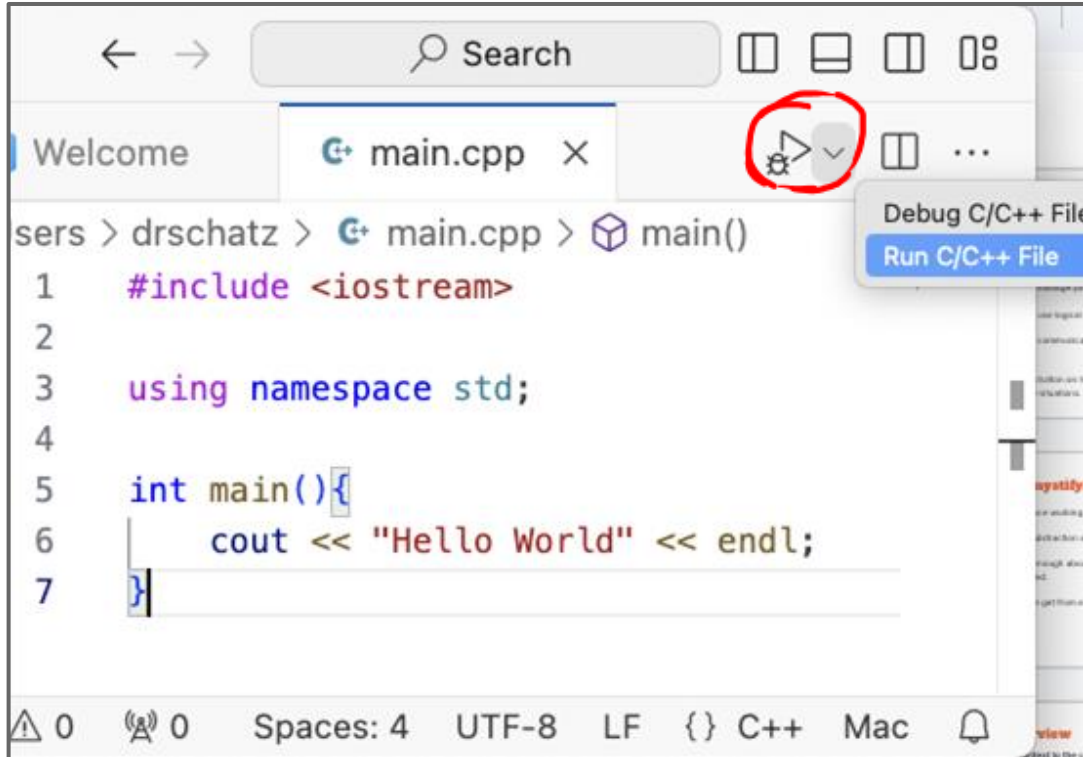
- 340 is about looking into some of the details that have been abstracted away that will help us be customize, debug, and understand computers better.

What should you remember?

- 340 is about looking into some of the details that have been abstracted away that will help us be customize, debug, and understand computers better.
- **This graphic is a simple representation of a computer system.**
 - **Application code** are things you interact with directly.
 - **OS code** helps your computer run and manage resources.
 - **All code** interacts with the hardware which is what actually executes logic.



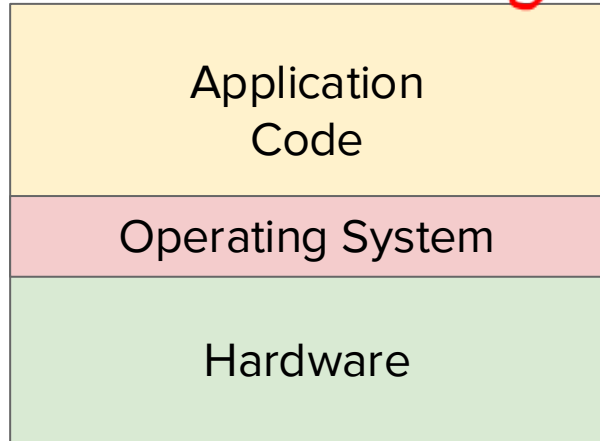
“Hello World” - High Level



```
1  #include <iostream>
2
3  using namespace std;
4
5  int main(){
6      cout << "Hello World" << endl;
7  }
```

"Hello World" - Medium Level

```
g++ hello.cc -o exec  
./exec
```



← Terminal

← compiler

1. using terminal
2. switch to compiler code
3. saves the exec to g++ memory
4. switch back to terminal

If we removed the operating system could we still run the exec executable we just made?

A) Yes

☒ B) No

C) Unsure

clicker.cs.illinois.edu

Q4

~Code~
340



If we deleted hello.cc but not exec could we still run the program to print hello world?

- ☒ A) Yes
- ☐ B) No
- ☐ C) Unsure

clicker.cs.illinois.edu

Q5

~Code~
340



Information in a Computer

Bit - 1 or 0

Byte - 8 bits

0100 0001

How many bytes are in this sequence of 16 bits?

0000 0110 0000 0111

A) 1

B) 2

C) 4

D) 16

E) Unsure

clicker.cs.illinois.edu

Q6

~Code~
340



What do these bits represent?

0100 0011

- A) 67
- B) C
- C) Part of a computer instruction
- D) Part of an internet packet
- ☒ E) I need more context

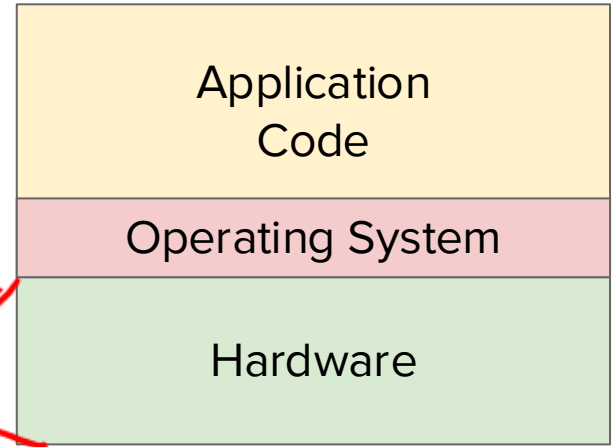
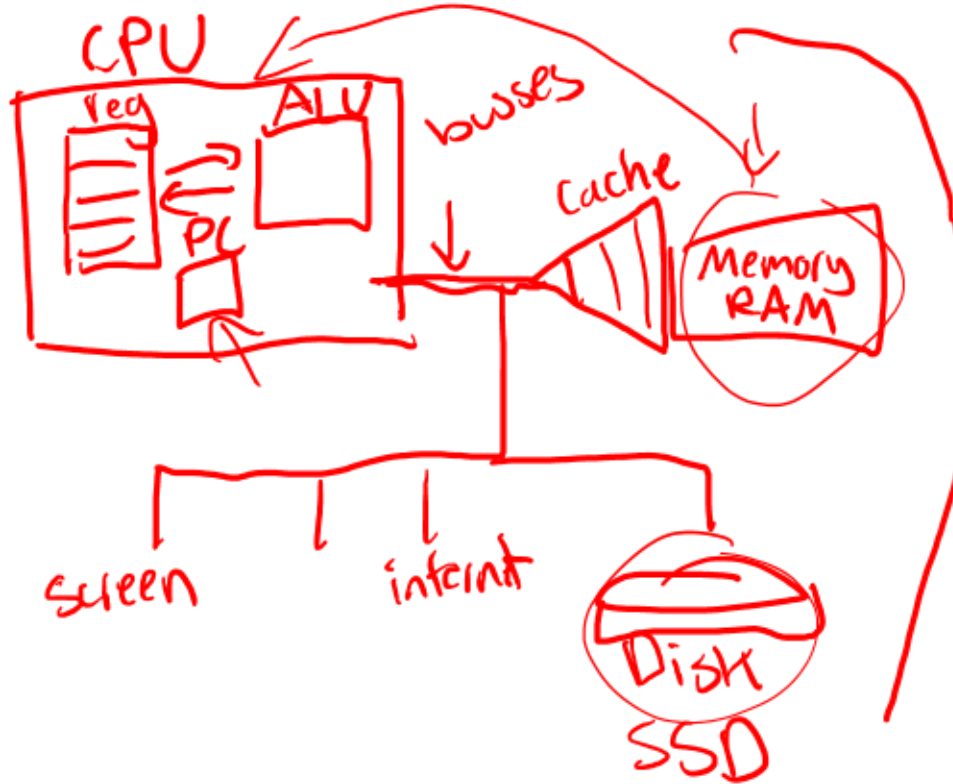
clicker.cs.illinois.edu

Q7

~Code~
340



Computer Hardware Diagram



“Hello World” - Low Level

→ g++ hello.cc -o exec
./exec

— keyboard input is converted to bytes then sent to the register

Super Low Level

Every step involves:

- AND, OR, NOT, XOR: Building blocks of all logic
- Multiplexers
- D Flip-Flops: Storing 1-bit state
- Clock signal: Synchronizing all activity

Super Super Low Level

Every action is controlled by:

- Binary voltage levels (e.g., $0V = 0$, $\sim 5V = 1$)
- A square wave oscillator
- Buses that carry multi-bit voltage patterns across the chip and board

What should you remember?

- There is a lot you don't YET know and a lot of details you may never care to know. Abstraction allows us to often ignore aspects we don't need to dive into.
- All code is run on hardware.
- The following terms exist - Compiler, application, CPU, PC, ALU, OS, memory, disk, I/O.
- The OS has a major role
 - Switches between running programs
 - Interfaces with many parts of the hardware like memory and other devices

CS 340 - Computer Systems

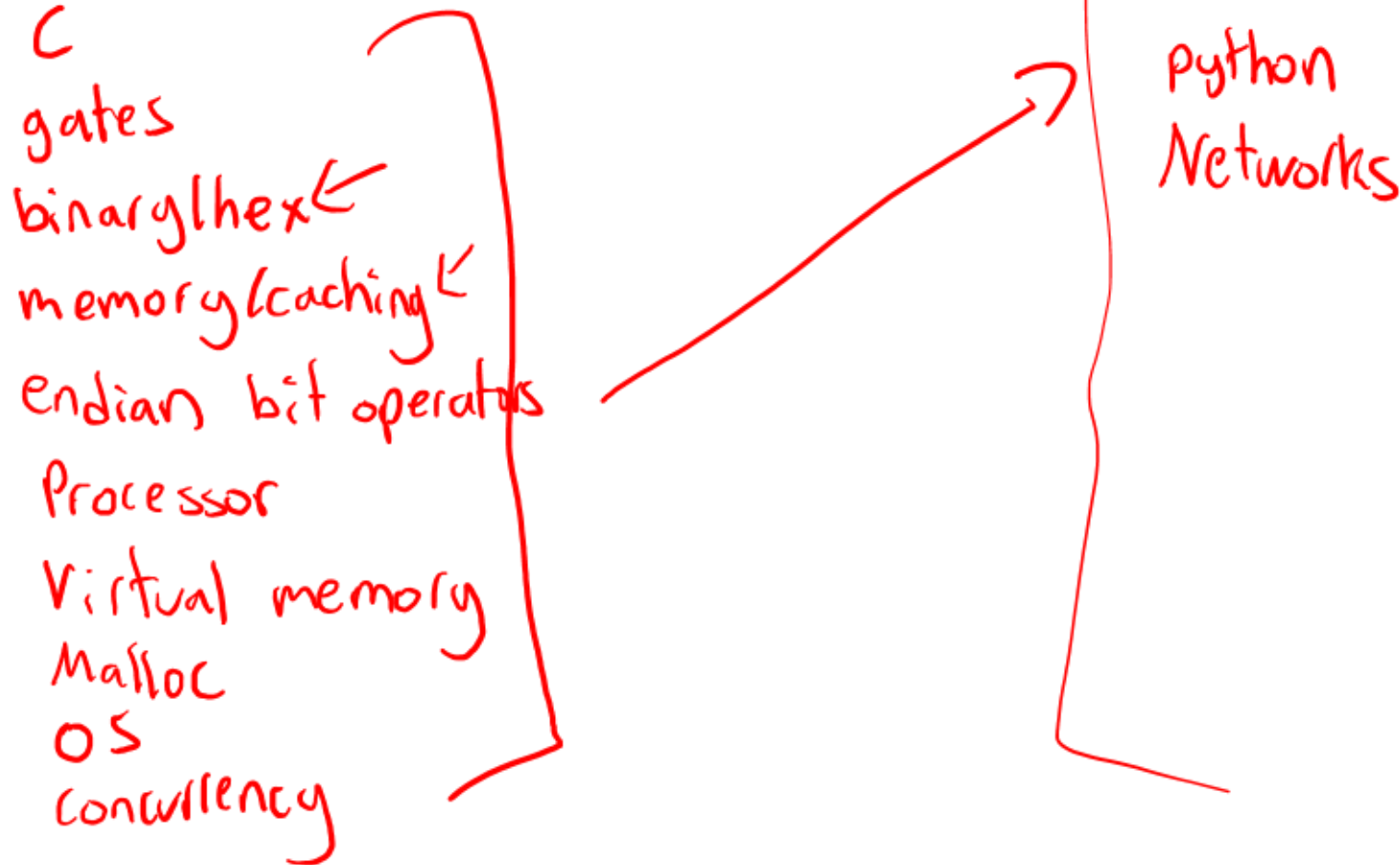
LG: Understanding, at a high level, how we get from electricity to a website.

LG: Developing mental models of how different computer systems work so that you can understand more complex ideas if needed later.

LG: Getting experience working on unstructured, substantial coding projects.

CS 340 - Part 1 Path

C
gates
binary/hex
memory/caching
endian bit operators
Processor
Virtual memory
Malloc
OS
concurrency



Part 2
python
Networks

How we will get there?

Lecture - Informational and practical skills

HW - conceptual and smaller tasks around topics learned in class.

MPs - practice working on unstructured coding projects related to systems and topics covered.

Exams - Covers lecture clickers, MPs and HW.

TODO

- Peruse the website and campus wire



- Write down important exam, MP, HW, final dates

- Start environment check-off ASAP ← sept 15



- Come meet Prof. Schatz and course staff at collab time today!



- Try out the C++ refresher course if you are feeling nervous about the programming aspect